Programmazione Dichiarativa

Giuseppe
Università di Parma
giuseppe.diblasi@studenti.unipr.it

A.A. 2026/2027

ABSTRACT

Appunti del corso di Programmazione Dichiarativa.

Indice

Indice

1 Lezione 1 del 21 settembre 2026 3
1.1 Modalità esame 3
1.2 Contenuti del corso 3
2 Introduzione al corso 4
2.1 Confronto tra reti neurali e AI simbolica 4
3 Ragionamento ad alto livello 5
3.1 Differenze AI sub simbolica e simbolica 5
3.2 Differenze tra Progammazione Imperativa e Dichiarativa 6
4 Lezione 2 del 28 settembre 2026 7
4.1 Richiami di logica del primo ordine 7

1 Lezione 1 del 21 settembre 2026

1.1 Modalità esame

Esame orale con domande di teoria più presentazione di un progetto.

1.2 Contenuti del corso

Il corso di Progammazione Dichiarativa si concentra sulla parte di intelligenza artificiale simbolica.

Quindi l'obiettivo è quello di saper modellare al meglio un problema di interesse in modo da farlo risolvere a un risolutore, detto **solver** per ottenere una soluzione corretta.

> Il solver è un software che permette di risolvere modelli o problemi che gli vengono forniti.
>
> Fornisce una soluzione corretta e dimostrabile per il problema da risolvere.

Il corso è molto simile a quello di Constraint Programming, infatti l'approccio è quello della programmazione dichiarativa invece di quella imperativa.

2 Introduzione al corso

Una delle domande poste a lezione dal prof è come classificare l'intelligenza di un sistema AI moderno, per esempio quello di un LLM.

Un LLM, attualmente, è in grado di generare codice di media qualità su progetti piuttosto contenuti in termini di complessità e scopo. Per progetti molto più complessi che richiedono codice di alta qualità è praticamente impossibile delegare ogni attività a un LLM.

Come possiamo dire che l'LLM abbia compreso i concetti per cui è stato addestrato e che non stia semplicemente ripetendo i pattern che è riuscito ad osservare? Stessa domanda può essere posta anche per un essere umano.

2.1 Confronto tra reti neurali e AI simbolica

Reti neurali

- Utilizzate dagli anni 90 in poi grazie alle prime GPU.
- Hanno come scopo quello di simulare la struttura e il comportamento del cervello umano.
- Hanno un approccio bottom-up, si parte dai neuroni (nodi) e si ottiene una rete.
- Al giorno d'oggi abbiamo LLM general purpose, con l'aumento delle prestazioni delle GPU aumentano anche le prestazioni della rete.

AI Simbolica

- Utilizzate fino agli anni 90.
- Sono dei dimostratori, infatti l'obiettivo è quello di theorem proving.
- Hanno un approccio top-down.
- Possiamo modellare la conoscenza del problema tramite **Knowledge representation (KR)**
- Possiamo parlare anche di **automated reasoning (AR)**
- È relegata a problemi più di nicchia.

Uno dei motivi per cui non si usa l'AI simbolica è dovuto al fatto che non è possibile parallelizzare la ricerca di soluzini per un dato modello dato che non girano su GPU. Questo è dovuto al fatto che la ricerca della soluzione per un dato modello avviene tramite una **catena di ragionamento sequenziale**; non è la stessa catena di ragionamento che usano alcune reti neurali odierne.

3 Ragionamento ad alto livello

Con la programmazione dichiarativa possiamo ragionare ad alto livello senza preoccuparci di dettagli implementativi. La conoscenza per un programma di AI simbolica è esplicita:

- Common Knowledge
- Specifiche
- Mancanza di conoscenza

Invece nel caso di una rete neurale, anche se i pesi sono in chiaro, non sono interpretabili da un essere umano.

Come possiamo vedere dall'albero di ricerca qui sotto, non è possibile parallelizzare la ricerca di una soluzione corretta poiché non siamo a conoscenza dell'intero albero, dato che i rami vengono scoperti man mano che si esplorano i nodi. Questo su un'architettura GPU comporterebbe una continua riallocazione dei worker e di risorse.

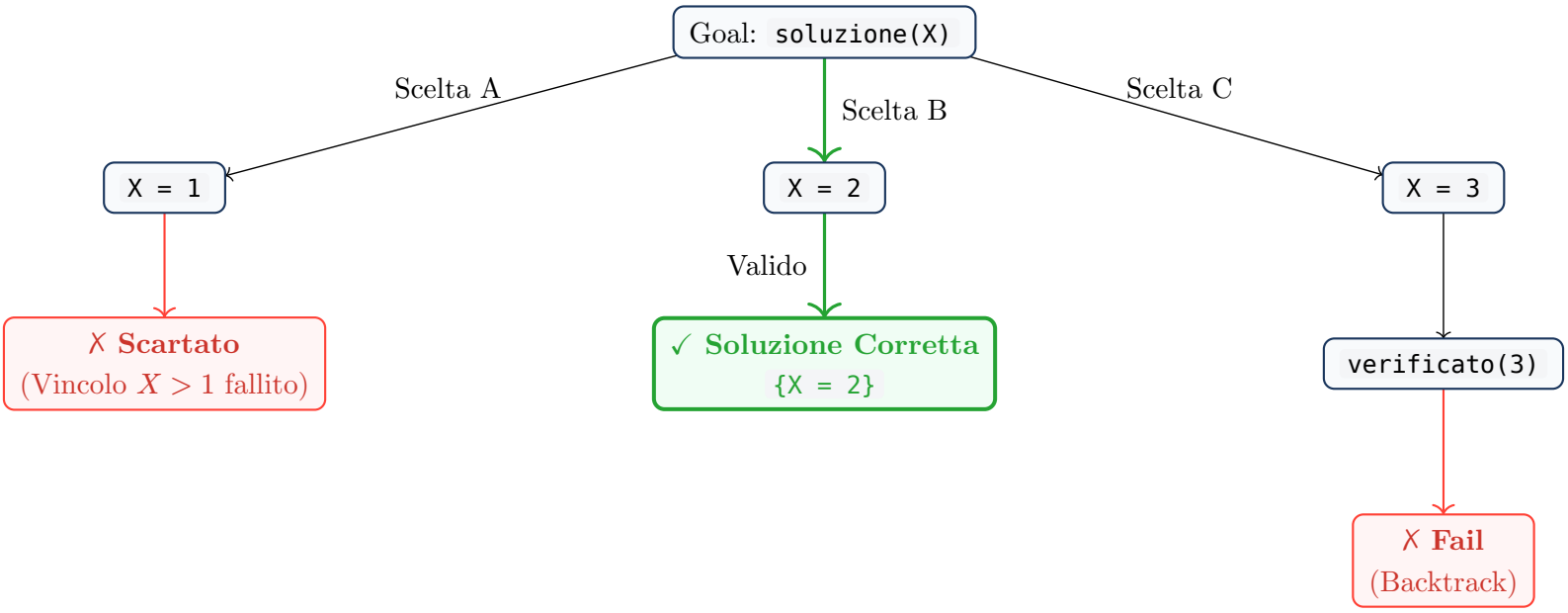


Figura 1. Albero di esplorazione del solver: rami scartati (rosso) e soluzione (verde)

3.1 Differenze AI sub simbolica e simbolica

Sub simbolica

- Lavora su enormi quantità di dati
- Poco significato dei dati (dati di basso livello, es. i colori dei pixel)
- Semantica dei dati: no
- Elevata velocità
- Non molto affidabile dal punto di vista tecnico
- Interpretabilità: no
- Black box: sì

Simbolica

- Poca quantità di dati

- Alto significato dei dati
- Semantica dei dati: sì
- Bassa velocità
- Altamente affidabile, la soluzione è sempre corretta (se il modello è corretto); il solver fornisce una dimostrazione della correttezza della soluzione.
- Interpretabilità: sì
- Black box: no

3.2 Differenze tra Progammazione Imperativa e Dichiarativa

Programmazione imperativa

- Specifiche: Spesso informali, e molto vicine all'implemetazione del codice vero e proprio.
- Modello algoritmico.
- Implementazione.
- Manutenzione: bug fixing ed evolutiva.

Progammazione dichiarativa

- Specifiche + **KR**: possono essere viste come un contratto matematico. Le specifiche sono praticamente il programma da risolvere.
- Goal (obiettivo): porta alla soluzione del problema grazie al **solver** che si occupa di **AR**
- Specifiche e Goal formano quello che si chiama **modello**.

4 Lezione 2 del 28 settembre 2026

Questa lezione si concentrerà su richiami di logica, utile per lo studio della programmazione dichiarativa. Verranno introdotti i concetti di predicati, costanti e verranno definiti anche formalmente.

4.1 Richiami di logica del primo ordine

Partiamo con un'introduzione formale:

Definizione 1 — Sintassi logica del primo ordine

Un linguaggio del primo ordine è caratterizzato in maneira univica da un alfabeto Σ costituito da:

- Un insieme $\prod$ di ***simboli di predicato***.
- Un insieme $\mathcal{F}$ di ***simboli di funzionee di costante***.
- Un insieme infinito numerabile $\mathcal{V}$ di simboli di variabile.

Ai simboli di Σ si associa la funzione di ***arità*** $\text{ar} : \Sigma \to \mathbb{N}$ tale che:

- $\text{ar}(X) = 0$ per ogni $X \in \mathcal{V}$
- $\text{ar}(f) \geq 0$ per ogni $f \in \mathcal{F}$
- $\text{ar}(p) \geq 0$ per ogni $p \in \prod$

Un simbolo $c \in \mathcal{F}$ è un simbolo costante se $\text{ar}(c) = 0$.